

50.043 Database Systems

Project Lab 4 Report

Class: CI01

Names: Tan Li Hon (1008920) & Celeste Chan Shi Ting (1008994)

Design Decisions

findLeafPage() method | BTreeFile.java

We used recursion to implement the findLeafPage() method in BTreeFile.java. The function traverses the internal nodes until it reaches a leaf page. At each internal node, we iterated the entries in order and the appropriate child is chosen based on key comparison. The design decision was to manage the handling of equal keys with duplicates. Duplicate keys can span multiple leaf pages thus we used LESS_THAN_OR_EQ during the comparison of search field to entry keys. Whenever $f \leq \text{entry.key}$, it will go left. This ensures leftmost leaf page that contains the key will always be returned.

splitLeafPage() and splitInternalPage() methods | BTreeFile.java

When a page is full, a new page will be created to the right of the existing page and half the entries will be moved to a new page using reverse iterator. It moves in a reverse order and is reinserted in forward order to preserve the sorted order of keys.

stealFromLeafPage() method | BTreeFile.java

stealFromLeafPage() evenly redistributes tuples between 2 leaf pages when 1 fall below minimum occupancy. The number of tuples to move is calculated using $\text{sibling.getNumTuples()} - \text{totalTuples}/2$ to ensure both pages end up as close to half full as possible. After moving tuples, the parent entry key will be updated to reflect the new first key of right page, maintaining B+ tree invariant that parent keys separate key ranges of child pages.

stealFromLeftInternalPage() and stealFromRightInternalPage() methods | BTreeFile.java

These methods redistribute entries between internal pages. The key design decision is key rotation pattern where keys are not moved directly between siblings but instead the parent key is pulled down into the deficient page and the sibling's boundary key is pushed up to replace it in the parent. Thus this maintains the B+ tree invariant that the internal page keys correctly separate their children's key ranges.

mergeLeafPages() method | BTreeFile.java

It moves all tuples from right leaf to left leaf page then updates sibling pointers to remove the right page from linked list and marks right page as empty for reuse. It is an inverse of splitLeafPage(), the corresponding parent entry is deleted using deleteParentEntry instead of copying a key up to the parent. This recursively handles the case where parent falls below minimum occupancy.

mergeInternalPages() method | BTreeFile.java

In addition to moving all entries from right to left page, the parent key entry must be pulled down to connect the 2 pages' children. Internal pages store routing keys thus without a pulled down key, there will be a gap between last left child and first right child. After merging, updateParentPointers() will be called to update the children that moved.

Non-Trivial Parts

findLeafPage() method to handle duplicate keys | BTreeFile.java

findLeafPage() method has to correctly handle duplicate keys, which may span multiple leaf pages after a split. We therefore use LESS_THAN_OR_EQ when comparing the search field with an internal entry key. When $f \leq \text{entry.key}$ it traverses to the left child. Thus, when the search key is equal to an internal entry key, the method continues left and returns the leftmost leaf page that may contain the key. This is important because duplicate keys can be distributed across multiple leaf pages. After a leaf page split, splitLeafPage() returns the newly created right page for the insertion, allowing subsequent duplicate-key insertions to spread toward the right while findLeafPage() continues to locate the leftmost matching page for searches. This design ensures that both duplicate-key searches and insertions behave correctly after leaf-page splits.

splitLeafPage() method for sibling pointer maintenance | BTreeFile.java

When leaf page is split, new page is inserted between original page and existing right sibling. Thus, this requires updating 4 sibling pointers to maintain doubly linked list of leaf pages. The pointers must be updated in the correct order. Firstly, the new page's right sibling is set to original page's old right sibling. Then, the new page's left sibling is set to the original page. Thirdly, if right sibling exists, the left sibling pointer is updated to point to new page instead of original page. Lastly, original page right sibling is updated to point to new page. The original, new page and right sibling must be added to dirtypages map immediately after modification to ensure that subsequent calls to getPage() will return updated version rather than cached version that will cause sibling pointer inconsistencies.

Key Rotation in stealFromLeftInternalPage() and stealFromRightInternalPage() | BTreeFile.java

Unlike leaf pages where tuples can be moved directly, internal pages need to rotate through the parent. The parent key is pulled down as the leftmost entry of the right page thus connecting the last child of left sibling to first child of right page. The last key of left sibling will then be pushed up to replace parent key. This rotation is done 1 entry at a time in a loop to satisfy the child pointer overlap constraint of insertEntry().

Pulling Down Parent Key in mergeInternalPages() | BTreeFile.java

Correctly pulling down parent key when merging internal page is a non-trivial part. When 2 internal page merge, a gap is formed between rightmost child of left page and leftmost child of right page. The parent entry key then helps to bridge this gap formed. The pulled-down entry will have lastLeft.getRightChild() as its left child and firstRight.getLeftChild() as its right child which correctly connects both subtrees. After all entries moved, updateParentPointers() will update the children that shifted from right page to point to the left page as its new parent.

Sibling Pointer Maintenance in mergeLeafPages() | BTreeFile.java

When merging 2 leaf pages, doubly linked list need to be updated. The rightPage is removed from the list by making leftPage.rightSibling = rightPage.rightSibling and rightSibling.leftSibling = leftPage. The right sibling page will also be added to dirtypages as it got modified.

API Changes

No changes made to the API.

Missing or Incomplete Elements

No missing or incomplete elements. All tests passed.